

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
Disciplina <i>Programação Orientada a Objetos</i>	
Nome do(a) acadêmico(a)	Valor 7,0
Professor <i>Sergio Souza Novak</i>	Turma <i>B</i>
Data da Avaliação <i>13/05/2026</i>	Código <i>ESW448</i>

ATENÇÃO: Somente serão passíveis de **REVISÃO** avaliações resolvidas à **TINTA**.

ORIENTAÇÕES PARA A RESOLUÇÃO

- A avaliação é individual, sem consulta;
- É permitida a utilização da IDE **IntelliJ IDEA** com a Linguagem **JAVA**;
- O código Java das questões de implementação deve ser preenchido **a caneta** na área retangular abaixo de cada questão;
- Todas as 10 questões possuem o mesmo peso;
- A interpretação de texto faz parte da avaliação;

QUESTÕES TEÓRICAS

INSTRUÇÕES: Esta seção contém **5 exercícios**. Recomenda-se reservar **40 minutos** para sua resolução. A avaliação é individual e **sem consulta**.

1. (**Ano: 2018 Banca: NC-UFPR Órgão: FOZPREV Prova: Analista Previdenciário**) Classes abstratas são um recurso importante na linguagem Java, pois incentivam o polimorfismo e dão flexibilidade à modelagem de classes, favorecendo a abstração. A respeito de classes abstratas, considere as seguintes afirmativas:

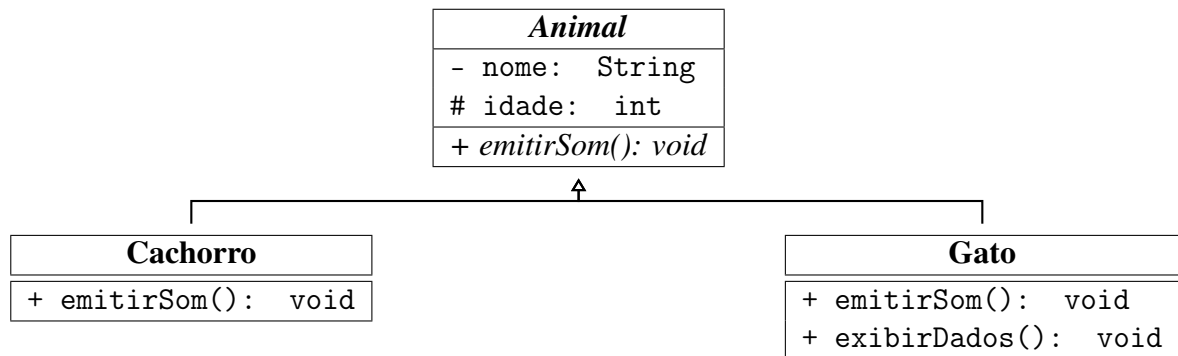
1. Classes abstratas não podem conter atributos.
2. Uma classe abstrata pode conter métodos concretos.
3. O construtor de uma classe abstrata deve ser abstrato.
4. Classes abstratas podem ser instanciadas.

Assinale a alternativa correta.

- A) Somente a afirmativa 1 é verdadeira.

- B) Somente a afirmativa 2 é verdadeira.
- C) Somente as afirmativas 1 e 3 são verdadeiras.
- D) Somente as afirmativas 1 e 4 são verdadeiras.
- E) Somente as afirmativas 2, 3 e 4 são verdadeiras.

2. Considere a seguinte estrutura de classes em Java que modela um sistema simples de animais:



```

public abstract class Animal {
    private String nome;
    protected int idade;
    public abstract void emitirSom();
}

public class Cachorro extends Animal {
    @Override
    public void emitirSom() { System.out.println("Au Au"); }
}

public class Gato extends Animal {
    @Override
    public void emitirSom() { System.out.println("Miau"); }

    public void exibirDados() {
        // Tentativa de acesso aos atributos da superclasse
    }
}
  
```

A respeito da estrutura de classes acima, descreva como os modificadores de acesso `private`, `protected` e `public` influenciam a visibilidade dos membros da classe `Animal` nas subclasses `Cachorro` e `Gato`. Em sua resposta, explique se os atributos `nome` e `idade` podem ser acessados diretamente no método `exibirDados()` e justifique sua resposta com base nos conceitos de encapsulamento e herança.

3. Considere o uso de polimorfismo no trecho de código Java abaixo, utilizando as classes `Animal`, `Cachorro` e `Gato` definidas anteriormente:

```
Animal meuPet = new Cachorro();  
meuPet.emitirSom();
```

Com base exclusivamente no conceito de **Sobrescrita de Métodos (Overriding)**, julgue as afirmativas a seguir como Verdadeiras (V) ou Falsas (F):

- () Para que ocorra a sobrescrita, o método na subclasse deve possuir o mesmo nome e a mesma assinatura (parâmetros) do método na superclasse.
 - () Um método definido como `final` na superclasse pode ser livremente sobrescrito pelas subclasses para especializar seu comportamento.
 - () O uso da anotação `@Override` é opcional, mas ajuda o compilador a verificar se a sobrescrita está ocorrendo corretamente.
 - () A sobrescrita permite que uma subclasse forneça uma implementação específica para um método que já foi definido em uma de suas superclasses.
 - () Se um método for declarado como `static` na superclasse, ele pode ser sobrescrito normalmente nas subclasses.
4. Suponha que a classe `Cachorro` possua um método exclusivo chamado `enterrarOsso()`, que não está presente na classe `Animal`. Considere o seguinte código:

```
Animal meuPet = new Cachorro();  
meuPet.emitirSom(); // Funciona (polimorfismo)  
// meuPet.enterrarOsso(); // Erro de compilação  
((Cachorro) meuPet).enterrarOsso(); // Funciona após o casting
```

Com base nesse cenário, responda:

- a) Por que o compilador não permite a chamada `meuPet.enterrarOsso()` sem o uso de *casting*, mesmo sabendo que o objeto foi instanciado como `new Cachorro()`?

- b) Explique o que ocorreria em tempo de execução caso a variável `meuPet` estivesse referenciando um objeto do tipo `Gato` e o programador tentasse realizar o *casting* para `Cachorro`:
`((Cachorro) meuPet).enterrarOsso();`.

5. O modificador `final` na linguagem Java é utilizado para restringir a alteração de elementos. Sobre o uso desse modificador em classes, métodos e atributos, analise as seguintes afirmações:

- I. Uma classe declarada como `final` não pode ser estendida, ou seja, ela não pode servir como superclasse para nenhuma outra classe.
- II. Um método declarado como `final` na superclasse pode ser sobrescrito nas subclasses, desde que a assinatura do método seja mantida.
- III. Quando aplicado a um atributo, o modificador `final` indica que o valor do atributo não pode ser alterado após ter sido inicializado (comportamento de constante).
- IV. Se uma classe for `final`, todos os seus métodos são implicitamente impedidos de serem sobrescritos, uma vez que a herança é proibida.

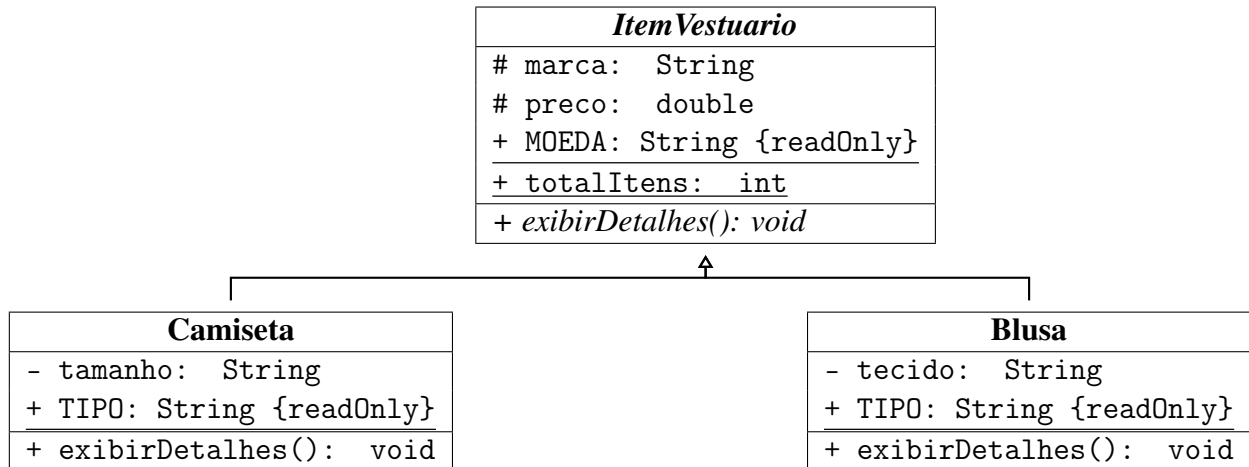
Assinale a alternativa **CORRETA**:

- A) Apenas as afirmações I, III e IV estão corretas.
- B) Apenas as afirmações I e IV estão corretas.
- C) Apenas as afirmações II e III estão corretas.
- D) Apenas as afirmações I e II estão corretas.
- E) Todas as afirmações estão corretas.

QUESTÕES PRÁTICAS

INSTRUÇÕES: Esta seção contém **5 exercícios** de implementação. Recomenda-se reservar **60 minutos** para sua resolução. A avaliação é individual e **sem consulta**. O código Java deve ser transcrito **a caneta** para as áreas retangulares reservadas abaixo de cada exercício.

Considere o seguinte diagrama de classes para as questões de 1 a 5:



LEGENDA: Atributos sublinhados indicam membros static e o rótulo `{readOnly}` indica membros final.

1. Implemente a classe abstrata `ItemVestuario` com seus atributos protegidos e os membros estáticos conforme o diagrama.

2. Implemente a classe concreta `Camiseta` realizando a herança de `ItemVestuario` e definindo seus membros específicos.



3. Implemente a classe concreta `Blusa` realizando a herança de `ItemVestuario` e definindo seus membros específicos.

4. Suponha que a variável `item`, declarada como `ItemVestuario`, esteja referenciando um objeto do tipo `Camiseta`. Sabendo que a classe `Camiseta` possui um método exclusivo chamado `dobrarCamiseta()`, demonstre o código necessário para realizar o *downcasting* e invocar esse método. Explique por que o compilador exige o uso de parênteses para essa operação.

5. Discorra 1 parágrafo sobre a importância das classes abstratas na aplicação dos princípios **SO-LID**.

